

Trabalho 1 (T1): Paralelização de uma aplicação com OpenMP

Unidade 03 | Grupos de até 3 integrantes

O trabalho tem quatro momentos: a apresentação deste enunciado e a formação dos grupos, uma aula de acompanhamento em laboratório em que o professor valida o problema escolhido, a entrega no Moodle (véspera da apresentação) e a **apresentação para a turma**. As datas estão no cronograma do Moodle.

1. Objetivo

Pegar uma aplicação sequencial existente, descobrir onde vale a pena paralelizar, instrumentá-la com diretivas OpenMP e **avaliar o desempenho da versão paralela** em uma máquina multicore.

O foco do trabalho não é escrever a aplicação: é a instrumentação e, principalmente, a **análise dos resultados**. Analisar resultados significa explicar o que causou os números obtidos, não repetir em texto o que o gráfico já mostra. Essa distinção é central na correção.

O trabalho integra três dimensões: as métricas de avaliação de desempenho (speed-up, eficiência, escalabilidade forte e fraca), o método de decidir o que paralelizar (perfil de execução e balanceamento de carga) e as construções do OpenMP (**parallel for**, escopo de variáveis, **reduction**, escalonamento). As seções 4 e 5 detalham o que é exigido de cada uma.

2. Escolha do problema

O grupo escolhe a aplicação. Pode ser um código pronto pego de um repositório, de um livro ou de um artigo, desde que o grupo **saiba explicar o que ele faz**. A escolha precisa ser validada pelo professor na aula de acompanhamento.

Critérios que a aplicação precisa atender

1. **Tempo sequencial suficiente.** Dimensione a entrada de referência para que a versão sequencial leve aproximadamente **de 30 a 60 segundos** na fase medida. Tempos muito curtos tornam a medição mais sensível ao overhead e à variação do sistema.
2. **Um eixo de paralelismo identificável.** Deve existir um laço, ou um conjunto de tarefas, que concentre a maior parte do tempo e cujas iterações sejam em boa medida independentes.
3. **Complexidade adequada.** Não deve ser trivial ao ponto de o **parallel for** resolver tudo em uma linha sem nada a analisar. Na dúvida, pergunte no acompanhamento.
4. **Entrada parametrizável.** O tamanho do problema precisa ser controlável por argumento de linha de comando ou constante, porque a escalabilidade fraca (seção 4.2) exige rodar com entradas diferentes.

Uso de ferramentas de IA

Estão liberadas ferramentas de apoio ao desenvolvimento e assistentes de IA. Duas condições: o grupo deve **saber explicar a solução resultante** linha a linha, e o relatório deve indicar **como a ferramenta foi usada**. Na apresentação, qualquer integrante pode ser questionado sobre qualquer parte do código.

3. Ambiente de execução

A execução é na **máquina do grupo** (notebook, desktop ou máquina do laboratório), com o maior número de núcleos disponível.

Todas as medições devem ser feitas na **mesma máquina**, com ela ociosa (sem navegador, compilação ou outro programa pesado competindo por núcleo).

Descrição da máquina (obrigatória no relatório)

Sem isso é impossível interpretar os resultados apresentados e relacioná-los à topologia da máquina, como a presença de SMT ou de núcleos heterogêneos:

- modelo do processador, frequência;
- número de **núcleos físicos** e número de **processadores lógicos disponíveis**;
- tamanho da cache L3 e quantidade de memória RAM;
- sistema operacional, versão do compilador e **flags de compilação** usadas.

`omp_get_num_procs()` informa quantos processadores estão disponíveis para o programa, normalmente processadores lógicos. Em Linux, comandos úteis são `lscpu`, `nproc`, `free -h` e `gcc --version`.

Regras de medição

1. **O tempo de referência sequencial (T_1) é o da melhor versão sequencial correta e equivalente.** Não é a versão paralela rodando com uma thread: essa carrega o custo das diretivas e infla o speed-up. As duas versões podem estar no mesmo executável compilado com `-fopenmp`, desde que T_1 venha do caminho ou da função realmente sequencial.
2. **Mesmas flags de otimização** (`-O2` ou `-O3`) na versão sequencial e na paralela. Comparar sequencial `-O0` com paralelo `-O3` não mede paralelismo, mede o compilador.
3. **Defina a região avaliada antes de medir.** Para calcular speed-up e eficiência, meça com `omp_get_wtime()` a fase computacional completa, incluindo tanto os trechos paralelizados quanto os que permanecem sequenciais. Use exatamente os mesmos limites nas duas versões. Leitura e escrita de arquivos podem ficar de fora, desde que sejam excluídas das duas versões e também do perfil usado na análise de Amdahl. O relatório deve dizer claramente o que ficou dentro da medição.
4. **O tempo isolado do trecho paralelizado é uma medida complementar.** Ele ajuda a diagnosticar o comportamento do código, mas não substitui o tempo da fase computacional completa no cálculo do speed-up e da eficiência.
5. **Valide o resultado antes de medir.** As versões sequencial e paralela devem produzir resultados equivalentes para as entradas e quantidades de threads usadas.
6. **Repita as medições.** Execute cada configuração pelo menos dez vezes. Reporte a mediana e o menor e o maior tempo observados.
7. **Fixe o número de threads** por `OMP_NUM_THREADS` (variável de ambiente) ou `omp_set_num_threads()`, e diga no relatório qual dos dois foi usado.

4. Medições exigidas

4.1 Escalabilidade forte

Use a entrada de referência definida na seção 2 e mantenha seu tamanho fixo. Execute a versão sequencial e a paralela com 2 threads e depois de 2 em 2 até o total de processadores lógicos disponíveis. Inclua explicitamente o número de núcleos físicos e o total de processadores lógicos, mesmo que algum deles não caia no passo de dois.

Para cada configuração, registre a mediana e o menor e o maior tempo observados. Calcule o speed-up e a eficiência.

4.2 Escalabilidade fraca

Use como ponto inicial a mesma entrada de referência da escalabilidade forte, mas aumente o trabalho total na mesma proporção do número de threads, mantendo aproximadamente constante o trabalho por thread. O que deve crescer proporcionalmente é o trabalho, não necessariamente o parâmetro de entrada. Declare e justifique a regra usada com base na complexidade do algoritmo.

Use a mesma série de threads da escalabilidade forte. Para cada configuração, registre o tamanho da entrada, a mediana e o menor e o maior tempo observados. Calcule a eficiência da escalabilidade fraca.

4.3 Apresentação dos resultados

Apresente uma tabela para cada experimento e, no mínimo, dois gráficos:

- **speed-up da escalabilidade forte** em função do número de threads, com a **reta do speed-up ideal** como referência;
- **eficiência** em função do número de threads, com uma curva para a escalabilidade forte e outra para a fraca.

Sempre que possível, coloque várias curvas no mesmo gráfico em vez de gastar espaço com um gráfico por curva. Toda figura precisa de legenda e de referência no texto.

5. O relatório

O relatório deve ter **no máximo duas páginas**, em **PDF**, incluindo as referências. Use papel A4, margens de pelo menos 1,5 cm, fonte 10 ou 11 e espaçamento simples. O uso de uma ou duas colunas fica a critério do grupo. O código-fonte não vai dentro dele: é entregue à parte, como descrito na seção 6.

Use um cabeçalho compacto com a identificação do trabalho, os integrantes e, quando aplicável, o link e o commit do repositório. Não inclua capa, sumário ou apêndices. Figuras e tabelas devem aparecer próximas do texto que as interpreta, com legendas, eixos e números legíveis.

O que o relatório precisa cobrir

Organize o conteúdo em subseções curtas. A **análise dos resultados deve ocupar a maior parte** do relatório.

1. **O problema e seu potencial de paralelização.** O que a aplicação calcula e qual eixo foi escolhido, entre os possíveis. A escolha deve vir de **medição, não de leitura do código**: levante com **gprof** ou **perf** o perfil da versão sequencial sobre a mesma fase usada no cálculo do speed-up. Reporte a fração do tempo concentrada no trecho paralelizado e use-a para estimar, na escalabilidade forte, o limite de Amdahl para a quantidade de threads usada e seu limite assintótico. Partes excluídas da medição não entram nesse limite.
2. **A modelagem paralela.** Qual é a unidade de trabalho, como ela foi dividida entre as threads, se as unidades de carga são **homogêneas** ou não, e o que atrapalhava o paralelismo (dependências, acumuladores compartilhados, seções críticas) e como foi contornado.
3. **As diretivas usadas.** Quais diretivas e cláusulas OpenMP, onde e por quê, referenciando os trechos por arquivo e linha (por exemplo, `jacobi.c:42`). Se houve otimização (mudança de escopo, troca de **atomic** por **reduction**, ajuste de granularidade), descreva o efeito medido.
4. **Ambiente e metodologia.** Resuma a máquina, o compilador, as flags, a região medida, as entradas e o protocolo de repetição.
5. **Balanceamento de carga.** Como a carga se distribuiu entre as threads e se algum escalonador diferente do padrão (**dynamic**, **guided**, **chunk** ajustado) foi testado. Se **static** foi mantido, justifique. Medição vale mais que suposição aqui.
6. **Análise dos resultados.** A parte central. Explique **o que causou** as curvas obtidas: por que o speed-up satura onde satura, por que a eficiência cai, como a topologia da máquina aparece nos resultados (SMT, núcleos heterogêneos ou ausência desses recursos), se há fração sequencial visível (Amdahl) e se a escalabilidade fraca se sustenta melhor que a forte. Quando houver SMT, identifique no gráfico o ponto em que ele passa a ser usado.
7. **Uso de ferramentas de IA**, se houve.

As tabelas devem apresentar, para cada configuração, a mediana e o menor e o maior tempo observados. Prefira texto corrido a listas de itens curtos, que picotam o argumento, e referencie toda figura e tabela no texto.

6. O código

Entregue de uma das duas formas, à escolha do grupo:

- um **arquivo .zip** enviado no Moodle junto do PDF; ou
- um **link para repositório git** (GitHub, GitLab), informado no cabeçalho do relatório.

Se optar pelo repositório, ele precisa estar **público**.

Em qualquer das duas formas, a entrega deve ser suficiente para **reproduzir e conferir os resultados apresentados**: o código-fonte, o **Makefile** ou a linha de compilação usada, o gerador ou arquivo de entrada, os tempos brutos de todas as execuções, com o número de threads e o tamanho da entrada de cada configuração, e os arquivos necessários para recalculas as métricas e reproduzir as tabelas e os gráficos. Inclua também um **README** curto com os comandos exatos de compilação e medição. Se faltar alguma dessas peças, o resultado não será considerado reprodutível.

Nomes de variáveis claros e comentários nos pontos não óbvios (por que uma variável é **firstprivate**, por que aquele **chunk**) contam na avaliação.

7. Apresentação

A apresentação será realizada em sala de aula, para a turma. Para receber a nota do trabalho, cada integrante deve estar presente e participar efetivamente da apresentação. Quem faltar ou não apresentar receberá nota zero, mesmo que o grupo tenha realizado a entrega.

A apresentação deve durar **no máximo 10 minutos**. Priorize a modelagem paralela e a análise dos resultados. Espere perguntas sobre decisões específicas do código.

8. Avaliação

A nota considera os itens abaixo. Cada um é avaliado em quatro níveis: atingiu plenamente, atingiu parcialmente, quase não atingiu, não atingiu. A exigência de presença e participação na apresentação é uma condição individual para receber a nota e antecede a aplicação destes critérios.

#	Item
1	O relatório está bem escrito e no formato solicitado
2	O problema escolhido é adequado para a aplicação de paralelismo e tem complexidade compatível com a proposta
3	As versões sequencial e paralela produzem resultados equivalentes
4	O grupo usou corretamente as diretivas OpenMP e explicou no relatório como foram usadas
5	O código foi entregue de forma acessível, permitindo reproduzir os resultados apresentados
6	O perfil de execução sustenta a escolha do trecho paralelizado, e a modelagem descreve o eixo e a homogeneidade das unidades de carga
7	A questão do balanceamento de carga foi abordada, incluindo a pertinência de outro escalonamento
8	A topologia da máquina foi relacionada aos resultados, incluindo o efeito de SMT quando disponível
9	O protocolo de medição foi seguido, as duas escalabilidades foram plotadas e a eficiência foi calculada
10	A análise explica as causas dos resultados e relaciona as curvas à fração sequencial, aos custos adicionais e à escalabilidade da aplicação
11	Participação do grupo nas aulas de acompanhamento no laboratório
12	Na apresentação, cada integrante demonstrou domínio do trabalho ao explicar o funcionamento do código, interpretar os resultados e responder às perguntas

9. Entrega

No Moodle, até a data indicada no cronograma: o relatório em **.pdf** e, se o grupo optou pelo **.zip**, o código junto. Quem entregar o código por repositório git envia só o PDF, com o link e o commit no cabeçalho. Um envio por grupo, com os nomes dos integrantes no cabeçalho.